

SIMATS ENGINEERING ASSESSMENT TOOL 2

**COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
CSA51**

COURSE CODE:

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I _Pusalapati Chandu (192372119)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Pchandu

Annexure A

1. Secure Password Storage in Web Applications

A web application stores user passwords and needs to protect them against database breaches.

- A. Explain why storing plaintext passwords is insecure.
 - B. Suggest a secure method for storing passwords and justify your choice.
-

2. Digital Signature for Document Authentication

A company wants to ensure that official documents sent electronically are authentic and unaltered.

- A. Explain how digital signatures can be used in this scenario.
 - B. Justify how this method ensures both integrity and non-repudiation.
-

3. Preventing Replay Attacks in Online Transactions

An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

- A. Explain how replay attacks occur.
 - B. Suggest a cryptographic technique to prevent such attacks and justify your answer.
-

4. Secure Communication over Public Wi-Fi

A user accesses sensitive information using a public Wi-Fi network without encryption.

- A. Identify the risks involved in this scenario.
 - B. Recommend a secure communication method and justify how it protects user data.
-

5. Data Integrity in Cloud Storage

An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

- A. Explain how hashing can be used to verify data integrity.
- B. Suggest an additional mechanism to enhance trust and justify your answer.

Question 1 — Secure Password Storage in Web Applications

SCENARIO: *A web application stores user passwords and needs to protect them against database breaches.*

Part A: Why storing plaintext passwords is insecure

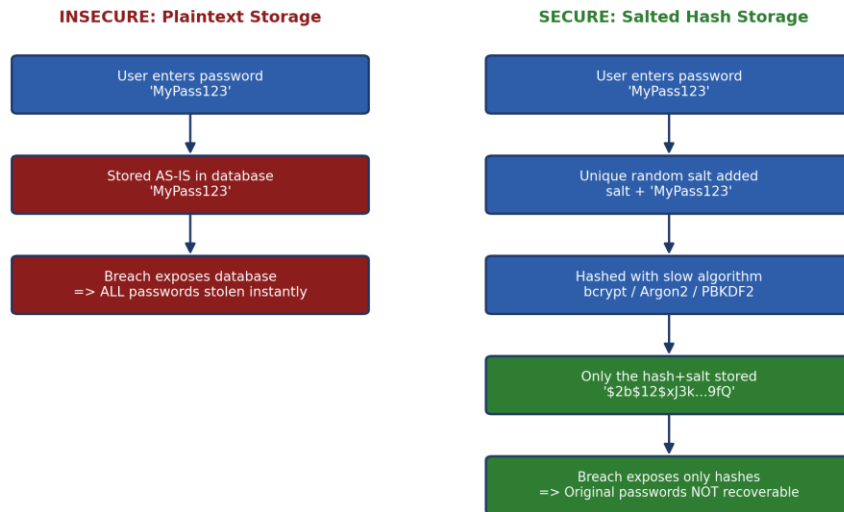
Storing passwords in plaintext means the database holds the user's exact password string with no transformation applied. This creates several serious risks:

- Total exposure on breach: if an attacker gains read access to the database (via SQL injection, stolen backup, insider access, etc.), every user's password is immediately readable with zero additional effort.
- Credential reuse impact: most users reuse passwords across services, so a single breach lets attackers compromise the same users' email, banking, and social media accounts elsewhere (credential-stuffing).
- No forensic delay: unlike encrypted or hashed storage, plaintext offers no computational barrier — there is nothing for the attacker to "crack", the password is already in final form.
- Insider risk: employees or anyone with database access (DBAs, support staff) can view user passwords directly, violating confidentiality even without an external breach.

Part B: Secure method for storing passwords, with justification

The recommended approach is to store a salted, slow-hashed representation of the password using a purpose-built password-hashing algorithm such as bcrypt, Argon2, or PBKDF2 — never a fast general-purpose hash like plain MD5 or SHA-256 alone.

- Hashing: the password is passed through a one-way cryptographic hash function, so the database stores only a fixed-length digest from which the original password cannot be feasibly reversed.
- Salting: a unique, random salt is generated per user and combined with the password before hashing. This ensures identical passwords produce different stored hashes, defeating precomputed rainbow-table attacks and revealing no password-reuse patterns across accounts.
- Deliberately slow / memory-hard algorithms: bcrypt and Argon2 are intentionally computation- and memory-intensive (with a configurable "work factor"), which makes brute-force and GPU/ASIC-based cracking attempts thousands of times slower than with a fast hash like SHA-256.
- Verification without exposure: on login, the server hashes the entered password with the stored salt and compares digests — the plaintext password is never persisted anywhere.

Password Storage: Plaintext vs. Salted-Hash Approach**Justification**

Even if the database is fully breached, the attacker only obtains salted hashes, not usable passwords, and cracking each one individually is computationally impractical at bcrypt/Argon2 cost settings. This directly neutralizes the breach scenario described, and is the industry-standard practice (used by OWASP-recommended implementations across virtually all modern web frameworks).

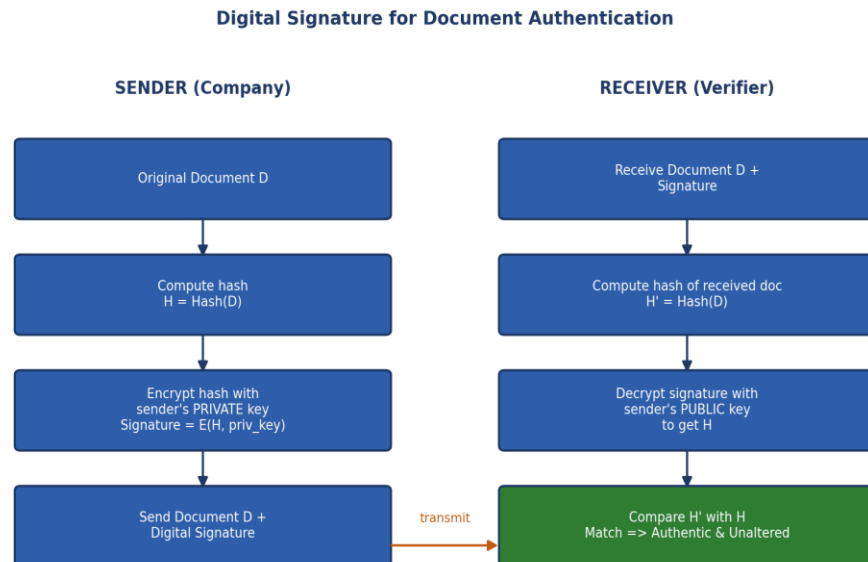
Question 2 — Digital Signature for Document Authentication

SCENARIO: A company wants to ensure that official documents sent electronically are authentic and unaltered.

Part A: How digital signatures can be used in this scenario

A digital signature binds a document to the sender's identity using asymmetric (public-key) cryptography. The process has two phases:

- **Signing (sender side):** the company computes a cryptographic hash of the document (e.g. using SHA-256), then encrypts that hash with the company's private key. This encrypted hash is the digital signature, attached to (or sent alongside) the original document.
- **Verification (receiver side):** the recipient independently computes the hash of the received document, then decrypts the received signature using the company's public key to recover the original hash. If the freshly computed hash matches the decrypted hash, the document is confirmed authentic and unmodified.



Part B: How this method ensures both integrity and non-repudiation

- **Integrity:** the hash is extremely sensitive to change — altering even a single character in the document produces a completely different hash value. If the document is tampered with in transit, the recomputed hash will not match the decrypted signature hash, and verification fails, immediately revealing the alteration.
- **Non-repudiation:** because the signature can only be produced using the company's private key (which only the company holds), a valid signature is proof that the company, and no one else, signed the

document. The company cannot later deny having issued it, since forging a valid signature without the private key is computationally infeasible.

- Authentication as a byproduct: successful verification with the company's public key also confirms the document genuinely originated from the company, not an impersonator.

Justification

Digital signatures give the company a legally and technically defensible mechanism: recipients can independently verify both who sent a document and that it was not altered, without needing a separate trusted channel for every exchange. This is why digital signatures underpin e-contracts, digitally signed PDFs, and code-signing certificates in real-world enterprise use.

Question 3 — Preventing Replay Attacks in Online Transactions

SCENARIO: *An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.*

Part A: How replay attacks occur

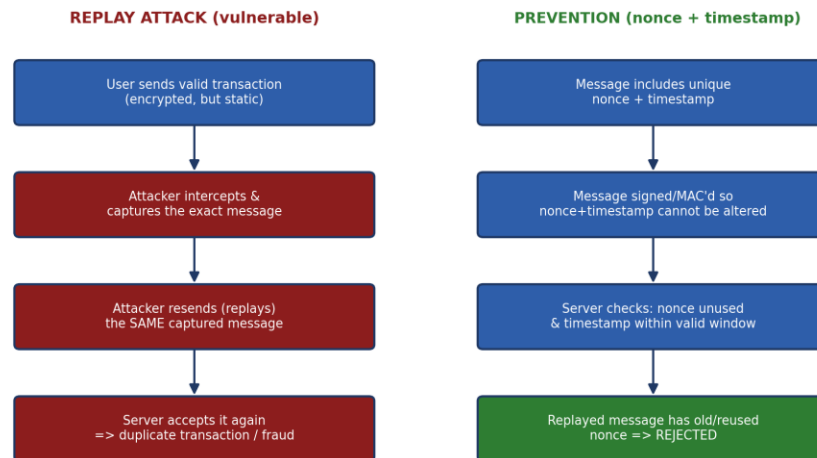
A replay attack does not require breaking encryption at all — it exploits the fact that a valid, correctly encrypted/authenticated message can simply be captured and resent later:

- An attacker passively intercepts a legitimate transaction message on the network (e.g. via packet sniffing on an unsecured link, or a compromised intermediary).
- Even though the message is encrypted or digitally signed, the attacker does not need to decrypt or forge it — they simply retransmit the exact same captured bytes to the server at a later time.
- If the server has no way of distinguishing this resent message from a fresh, legitimate one, it processes the transaction again — potentially duplicating a payment, transferring funds a second time, or re-authenticating a stale session.

Part B: Cryptographic technique to prevent replay attacks, with justification

The standard defense is to add a nonce (a number used once) and/or timestamp to every transaction message, and bind it cryptographically so it cannot be stripped or altered:

- Nonce: a unique, unpredictable, single-use value generated per transaction. The server maintains a record of recently used nonces and rejects any message reusing one — a replayed message inherently carries an already-seen nonce.
- Timestamp with a validity window: the message includes the time it was created; the server only accepts transactions within a short acceptable delay (e.g. a few seconds), rejecting anything older, which limits how long a captured message stays useful to an attacker.
- Sequence numbers: an incrementing counter per session (as used in TLS record protocol and Kerberos) ensures messages must arrive in strict order, so a resent earlier message is immediately recognized as out of sequence.
- Message Authentication Code (MAC) / digital signature over the nonce+timestamp: this is essential — the nonce and timestamp must be covered by the same signature/MAC as the transaction data, so an attacker cannot simply swap in a fresh nonce onto an old captured message.

Replay Attack vs. Nonce/Timestamp-Based Prevention**Justification**

Combining a signed nonce and timestamp means every transaction is cryptographically unique and time-bound. Even though the attacker still possesses a perfectly valid, correctly encrypted captured message, the server can positively identify it as a duplicate/expired transaction and reject it — this is exactly the mechanism used in protocols like TLS, Kerberos, and EMV chip-and-PIN payment authorization.

Question 4 — Secure Communication over Public Wi-Fi

SCENARIO: *A user accesses sensitive information using a public Wi-Fi network without encryption.*

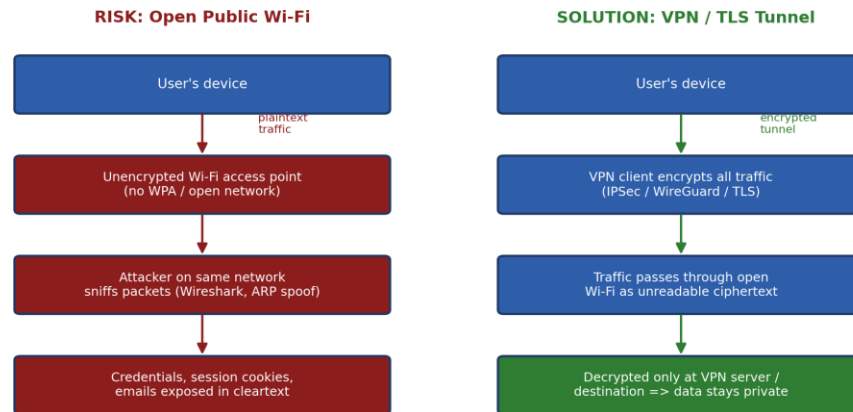
Part A: Risks involved in this scenario

- Eavesdropping / packet sniffing: on an open (unencrypted) Wi-Fi network, any device within range can capture all transmitted traffic using freely available tools (e.g. Wireshark), exposing usernames, passwords, and message content sent in cleartext.
- Man-in-the-Middle (MITM) attacks: an attacker can position themselves between the user and the access point (e.g. via a rogue "evil twin" hotspot or ARP spoofing) to intercept, read, and even modify traffic in transit.
- Session hijacking: captured session cookies or authentication tokens can be reused by an attacker to impersonate the user on websites without ever knowing their password.
- DNS spoofing / malicious redirection: an attacker controlling the network can redirect the user to fake login pages designed to harvest credentials (phishing at the network level).
- Malware injection: unencrypted downloads or software update channels can be tampered with in transit to deliver malicious payloads.

Part B: Recommended secure communication method, with justification

Route all traffic through a VPN (Virtual Private Network) using a strong protocol such as WireGuard, IPsec, or OpenVPN, and additionally ensure individual services are accessed only over TLS/HTTPS.

- End-to-end encryption of the tunnel: the VPN client encrypts all outbound traffic before it ever reaches the public Wi-Fi access point, so anyone sniffing the airwaves only sees unreadable ciphertext, regardless of how insecure the underlying Wi-Fi is.
- Authenticated tunnel endpoint: the VPN connection is established with the VPN server's verified identity (certificate-based), preventing MITM attackers on the local network from silently substituting themselves as the endpoint.
- Protection independent of the Wi-Fi network's own security: since the encryption happens at the VPN/TLS layer rather than relying on the Wi-Fi's (often absent or weak) WPA encryption, the user is protected even on a completely open hotspot.
- Complementary layer — HTTPS/TLS: even with a VPN, ensuring individual websites use HTTPS provides defense-in-depth, so data remains encrypted end-to-end to the actual service, not just to the VPN server.

Public Wi-Fi Risk vs. VPN/TLS Protection**Justification**

A VPN converts an inherently untrusted public network into a secure encrypted channel: interception on the local Wi-Fi segment yields only ciphertext, session hijacking via sniffed cookies becomes infeasible, and rogue-hotspot MITM attempts are defeated by certificate-based tunnel authentication. This is why VPN use is standard security guidance for anyone accessing sensitive data on public networks (airports, cafes, hotels).

Question 5 — Data Integrity in Cloud Storage

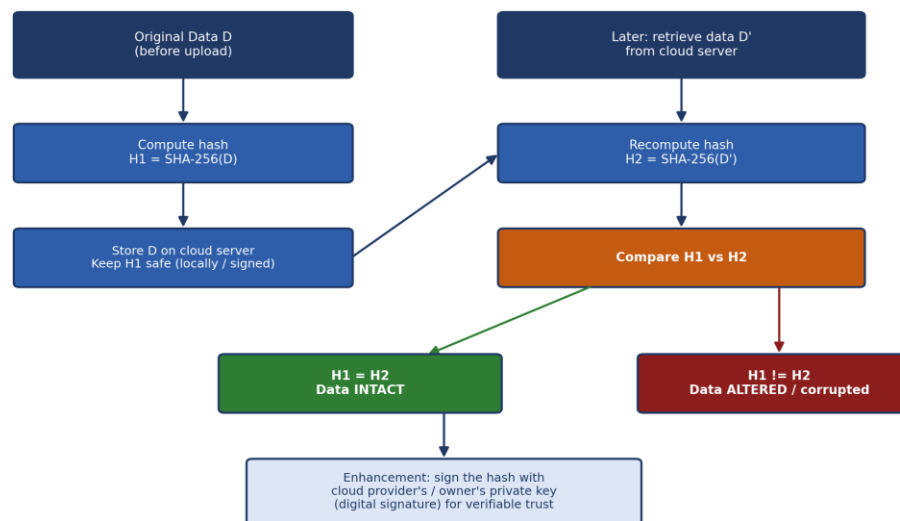
SCENARIO: An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

Part A: How hashing can be used to verify data integrity

Cryptographic hash functions (e.g. SHA-256) convert data of any size into a fixed-length digest with two key properties: the same input always produces the same output, and even a tiny change in input produces a drastically different, unpredictable output (the avalanche effect).

- Before uploading, the organization computes a hash $H1 = \text{Hash}(\text{Data})$ of the original file and stores this hash value securely and separately from the cloud copy (e.g. locally or in an access-controlled system).
- At any later point, the organization downloads the data from the cloud and recomputes its hash, $H2 = \text{Hash}(\text{Downloaded Data})$.
- If $H1 = H2$, the data is provably unchanged, byte-for-byte, since it was uploaded. If $H1 \neq H2$, even a single-bit modification — accidental corruption or malicious tampering — is immediately detected.
- This check is efficient: comparing two short fixed-length hash values is far cheaper than comparing entire large files directly.

Data Integrity Verification in Cloud Storage using Hashing



Part B: Additional mechanism to enhance trust, with justification

Digitally sign the hash (or use a Message Authentication Code, or a blockchain-based / third-party timestamping ledger) rather than relying on hashing alone.

- Digital signature over the hash: the organization signs $H1$ with its private key before storing/publishing it. This prevents an attacker who compromises both the cloud data AND the stored hash from recalculating a matching fake hash for tampered data — since they cannot forge a valid signature without the private key.

- HMAC (keyed hash): using a Hash-based Message Authentication Code with a secret key ensures that only parties holding the key can generate a valid integrity tag, preventing an attacker from simply recomputing a new hash after altering the data if they lack the key.
- Third-party timestamping / blockchain anchoring: publishing the hash to an immutable, independently verifiable ledger creates a trusted, tamper-evident record of the hash's existence at a specific point in time, which is useful for compliance, auditing, and disputes with the cloud provider itself.

Justification

Plain hashing alone verifies integrity only if the reference hash itself is trustworthy and unaltered. Layering a digital signature (or HMAC / independent timestamping) on top of the hash protects against the case where an attacker — including potentially a malicious insider at the cloud provider — modifies both the data and its stored hash together. This combination is exactly how integrity is enforced in real systems such as software package repositories, TLS certificate chains, and audit-compliant document archival systems.

Summary of Recommended Solutions

- Q1 (Passwords): Never store plaintext — use salted, slow hashing (bcrypt/Argon2/PBKDF2).
- Q2 (Document Authentication): Digital signature (hash + private-key encryption) ensures integrity and non-repudiation.
- Q3 (Replay Attacks): Signed nonce + timestamp/sequence number makes each transaction unique and time-bound.
- Q4 (Public Wi-Fi): Use a VPN (encrypted tunnel) plus HTTPS/TLS for end-to-end protection.
- Q5 (Cloud Integrity): Hash comparison detects tampering; digital signature/HMAC on the hash adds verifiable trust.